



BRANDFY

Brandfy — Documentação completa · v1.1.0

Estratégia de marca em artefatos reproduzíveis

Guia do usuário e referência técnica para criar, documentar e auditar sistemas de marca.

Documentação completa

Edição 1.1.0

Gerado em 14/08/2026

Fonte editorial: docs/

Sumário

Documentação do Brandfy	6
Fontes oficiais	6
Guia completo do usuário	7
Leia online ou como parte do ebook	7
Percurso recomendado	7
O resultado no repositório	8
Instalação do Brandfy	9
Requisitos	9
Instale com o CLI do Brandfy	9
Confira o catálogo	9
Atualize o CLI e a biblioteca	10
Como o Brandfy organiza uma marca	11
O papel de cada diretório	11
As camadas da informação	11
As quatro fases	11
As skills como especialistas	12
Primeira marca com o Brandfy	13
Prepare o projeto	13
Preencha a configuração	13
Escolha a entrada adequada	13
Confira a primeira estrutura	14
Entrevista e diagnóstico	15
Inicie a entrevista	15
Valide a cobertura	15
Diagnostique uma marca existente	15
Leia o resultado com os responsáveis	16
Construção do sistema de marca	17
Use o fluxo completo	17
Estratégia, naming e slogan	17
Voz e tom	17
Identidade visual e logo	18
Gates de aprovação	18

Logos, webfontes, tokens e templates	19
Exporte o sistema de logo	19
Prepare as webfontes	19
Gere os design tokens	19
Instale templates de canais	20
Manual editável e PDF da marca	21
O Markdown é a fonte	21
Compile o guia	21
Abra o resultado	21
Diferencie os dois PDFs	22
Auditoria da marca	23
Execute a conferência automatizada	23
Confronte definição, fonte e aplicação	23
Interprete o relatório	23
Uso da marca por agentes	24
Dê ao agente uma fonte única	24
Escreva instruções verificáveis	24
Preserve ativos aprovados	24
Confira a exportação	24
Solução de problemas	25
A skill não aparece	25
O setup alteraria arquivos	25
A entrevista não passa no check	25
O PNG não foi gerado	25
A webfont não carrega	26
O contraste foi reprovado	26
O PDF ficou sem imagens ou fontes	26
A auditoria ainda apresenta avisos	26
Dados para um relato de falha	26
Guia de desenvolvimento	27
Autoridades técnicas	27
Validação rápida	27
Arquitetura do Brandfy	28
Biblioteca instalada no projeto consumidor	28
CLI e gerenciador de skills	28

Tipos de skill	28
Limites de escrita	29
Dependências externas	29
Contrato das skills	30
Estrutura mínima	30
Protocolo operacional	30
Metadados para o agente	30
Referências e arquivos-base	30
Compatibilidade com o Skill Creator	31
Contrato dos artefatos	32
Configuração	32
Arquivos de trabalho	32
Arquivos finais	32
Fontes e derivados	33
Geradores e interfaces de linha de comando	34
Setup	34
Entrevista	34
Logos	34
Design tokens	34
Webfontes	35
Templates	35
Manual em PDF	35
Auditoria	36
Testes e validação	37
Suíte principal	37
Cobertura atual	37
Validador estrutural	37
Documentação e ebook	38
Release	38
Inspeção humana	38
Contribuição e manutenção	39
Prepare a alteração	39
Crie ou altere uma skill	39
Documente o comportamento	39
Valide antes da publicação	39

Versione a release	40
--------------------------	----

Documentação do Brandfy

O Brandfy organiza a construção de uma marca em skills especializadas. A documentação possui dois percursos porque o uso da biblioteca e a manutenção do repositório exigem informações diferentes.

PERCURSO	PARA QUE SERVE	COMECE POR
Guia do usuário	Instalar as skills, preparar um projeto e produzir os arquivos da marca	Guia completo do usuário
Guia técnico	Entender a arquitetura, alterar skills, manter geradores e publicar uma versão	Guia de desenvolvimento

O [guia do usuário](#) acompanha uma marca desde a instalação até a auditoria. Ele e o guia técnico compõem o PDF e o EPUB publicados em `ebooks/`, conforme a ordem registrada em `reading-order.txt`.

O [guia técnico](#) explica o contrato de uma skill, os diretórios gerenciados, os geradores, os testes e a manutenção da biblioteca. Ele deve ser usado junto com o código, que continua sendo a fonte executável do comportamento.

Fontes oficiais

As páginas descrevem quatro fontes que cumprem papéis diferentes:

- `skills/` implementa as especialidades e contém scripts, referências e arquivos-base instaláveis.
- `.brandfy/` registra configuração, briefing e evidências do trabalho em andamento.
- `brand/` guarda o manual e os ativos finais de uma marca.
- `tests/` e `scripts/validate.mjs` comprovam os contratos automatizados do repositório.

Quando a documentação divergir de uma interface executável, corrija a página e o teste relacionado na mesma alteração.

Guia completo do usuário

Uma marca costuma começar com material espalhado: um logo usado no site, cores copiadas de apresentações, fontes escolhidas por hábito e explicações que somente uma parte da empresa conhece. O Brandfy reúne esse material, conduz as definições que ainda faltam e grava um sistema que pessoas e agentes conseguem consultar antes de criar uma nova peça.

O Brandfy é uma biblioteca de skills para agentes. Ele não substitui a conversa com os responsáveis pela marca, a pesquisa jurídica nem a revisão de um designer. As skills ajudam a registrar evidências, comparar alternativas, produzir arquivos repetíveis e mostrar o que ainda precisa de aprovação humana.

Leia online ou como parte do ebook

Este percurso e a referência técnica são publicados no mesmo PDF e EPUB. Os arquivos Markdown em `docs/` são a fonte editorial, por isso uma correção feita aqui entra nas duas edições no próximo build.

- O PDF preserva a diagramação e funciona bem para leitura, compartilhamento e impressão.
- O EPUB permite ajustar a fonte e o tamanho em leitores digitais.

Baixe o PDF da documentação completa ou o EPUB da documentação completa. A pasta dos ebooks registra a edição vigente e os hashes dos artefatos.

Percurso recomendado

Comece pela instalação e confirme que as skills aparecem no projeto. Depois, leia como o Brandfy organiza os arquivos antes de executar o setup. Essa ordem evita que `.brandfy/` seja confundido com a pasta pública `brand/`.

O trabalho completo segue esta sequência:

1. Instale e confira as skills.
2. Entenda os arquivos e as camadas de informação.
3. Prepare o projeto consumidor.
4. Conduza a entrevista e o diagnóstico.
5. Construa ou revise o sistema de marca.
6. Gere logos, tokens, webfontes e templates.
7. Compile o manual e o PDF da marca.
8. Audite os arquivos antes de publicar.
9. Oriente outros agentes a usar a marca.
10. Investigue falhas conhecidas.

Uma marca existente pode começar pelo diagnóstico e preservar tudo o que já possui evidência de uso e aprovação. Uma marca nova precisa de entrevista antes da direção verbal e visual, pois o agente não deve completar uma lacuna com uma resposta apenas plausível.

O resultado no repositório

Ao final, o projeto consumidor mantém a origem do trabalho em `.brandfy/` e os arquivos utilizáveis em `brand/`. Um conjunto completo pode conter estratégia, voz, logos vetoriais e raster, favicons, paleta, tokens CSS e JSON, tema Tailwind, webfontes, templates, regras de acessibilidade, licenças, manifesto e manual em PDF.

O relatório `.brandfy/audit.md` fecha o percurso ao confrontar o manual com os arquivos produzidos. A aprovação automática não dispensa a abertura do PDF, dos logos e de pelo menos uma aplicação de cada canal no tamanho final.

Instalação do Brandfy

Requisitos

O repositório de destino precisa estar acessível ao agente e usar um arquivo `AGENTS.md` que possa receber o bloco gerenciado do Brandfy. O CLI e os geradores usam Node.js 22.20 ou posterior. Algumas saídas exigem ferramentas adicionais:

FERRAMENTA	USO
ImageMagick	Exportação raster de SVG para PNG
Pandoc	Conversão do manual Markdown para HTML
WeasyPrint	Geração do PDF a partir do HTML

As skills continuam úteis sem todos esses programas, mas o gerador correspondente informará a dependência ausente. Instale somente ferramentas compatíveis com o sistema operacional e com a política do repositório.

Instale com o CLI do Brandfy

Abra o terminal na raiz do projeto que receberá a marca e execute:

```
npm install --global @promovaweb/brandfy
brandfy init .
```

O CLI usa o gerenciador `skills` como dependência e instala o catálogo do Brandfy para o Codex. Também é possível executar a versão mais recente sem instalação global:

```
npx @promovaweb/brandfy init .
```

Depois do setup, confira os arquivos esperados:

```
brandfy doctor .
```

O Hub da Promovaweb instala a biblioteca externa em `.agents/skills/`, enquanto `.codex/skills/` permanece reservado às skills nativas do próprio Hub.

Confira o catálogo

Liste o conteúdo remoto antes da instalação quando precisar escolher apenas algumas especialidades:

```
brandfy skills list
```

Depois da instalação, confirme que o diretório contém `brandfy-setup`, `brandfy-builder` e as especialidades que serão usadas. A ausência de uma skill impede o fluxo coordenado de concluir a etapa relacionada, mas não invalida as demais skills instaladas.

Atualize o CLI e a biblioteca

O framework e o CLI usam a mesma SemVer. Atualize o pacote pelo npm e depois reinstale as skills do Brandfy registradas no projeto:

```
npm install --global @promovaweb/brandfy@latest  
brandfy update .  
brandfy doctor .
```

O `skills add` mantém a origem em `skills-lock.json`. Uma skill modificada diretamente pode divergir do repositório e deve ser preservada em outro diretório antes da atualização.

O gerenciador continua disponível diretamente quando for necessário escolher outro agente ou uma parte do catálogo:

```
npx skills add promovaweb/brandfy --agent codex --skill '*' -y --copy
```

Quando o diagnóstico informar estrutura ausente, siga o capítulo Primeira marca para preparar o projeto.

Como o Brandfy organiza uma marca

O papel de cada diretório

O Brandfy separa o material de pesquisa dos arquivos aprovados. Essa separação permite que uma entrevista registre uma hipótese sem apresentá-la no manual como definição final.

CAMINHO	CONTEÚDO	PODE SER PUBLICADO
<code>.brandfy/</code>	Configuração, briefing, entrevistas, evidências, relatórios e trabalho em andamento	Não, salvo escolha explícita do responsável
<code>brand/</code>	Manual, logos, fontes, tokens, templates e arquivos finais	Sim, depois da auditoria
<code>.agents/skills/brandfy-*/</code>	Método, referências, scripts e arquivos-base instalados	Não como parte da marca

O caminho de saída é configurável em `.brandfy/config.yaml`, embora `brand/` seja o padrão e o formato esperado pelas skills. Uma mudança de diretório precisa ser refletida nos comandos, links e relatórios.

As camadas da informação

Uma entrevista de marca mistura lembranças, preferências e afirmações comprovadas. O Brandfy registra cada uma na camada adequada:

- **Fato:** afirmação confirmada por uma fonte identificada.
- **Evidência:** arquivo, pesquisa, relato ou comportamento que fundamenta uma afirmação.
- **Interpretação:** leitura construída a partir de fatos informados.
- **Hipótese:** explicação que ainda precisa de teste.
- **Preferência:** gosto ou direção desejada, com seu motivo.
- **Escolha aprovada:** definição aceita por um responsável identificado.
- **Pendência:** pergunta ou trabalho que continua aberto.

Essa classificação aparece no JSON da entrevista, nos relatórios e no raciocínio das skills. Uma preferência visual pode orientar as rotas apresentadas, mas não deve ser descrita como preferência comprovada do público sem pesquisa.

As quatro fases

O fluxo completo percorre descoberta, definição, desenvolvimento e publicação. Cada fase produz uma fonte que a próxima consegue ler.

Na descoberta, a entrevista e o diagnóstico registram o que existe, o que foi dito e onde há conflito. Na definição, estratégia, naming, slogan e voz explicam a marca em linguagem verificável. O desenvolvimento transforma essas definições em sistema visual e ativos. A publicação compila o manual e confere os arquivos com a auditoria.

As skills como especialistas

`$brandfy-builder` coordena o percurso, mas não substitui as especialidades. Ele encaminha a tarefa para a skill adequada e retoma o plano depois que a etapa produz os arquivos esperados. Uma chamada direta continua válida quando o escopo é restrito, como recompilar tokens a partir de uma paleta já aprovada.

Cada skill precisa informar a fonte consultada, o arquivo alterado, a conferência executada e o que permaneceu aberto. O relatório final deve permitir que outro agente retome o trabalho sem inferir o estado anterior.

Primeira marca com o Brandfy

Prepare o projeto

Depois de instalar as skills, execute o setup na raiz do projeto consumidor:

```
node .agents/skills/brandfy-setup/scripts/setup.mjs --project .
```

O setup cria `.brandfy/`, prepara `brand/` e inclui um bloco delimitado no `AGENTS.md`. O conteúdo que já existe fora dos marcadores `brandfy:consumer:start` e `brandfy:consumer:end` deve permanecer intacto.

Execute novamente em modo de conferência:

```
node .agents/skills/brandfy-setup/scripts/setup.mjs --project . --check
```

O segundo comando comprova a idempotência e informa qualquer arquivo obrigatório que esteja ausente. Abra o diff do Git antes de avançar para confirmar que a escrita ficou restrita aos caminhos previstos.

Preencha a configuração

O arquivo `.brandfy/config.yaml` identifica o projeto e a pasta da marca. Uma configuração inicial usa esta forma:

```
project_name: Minha Marca
brand_directory: brand
mode: greenfield
language: pt-BR
primary_audience: público ainda em pesquisa
status: draft
```

Use `mode: greenfield` para uma marca nova. Uma identidade existente pode usar o modo de revisão adotado no projeto e começar pelo diagnóstico. Mantenha `status: draft` enquanto definições importantes ainda estiverem abertas.

Escolha a entrada adequada

Quando responsáveis, público, oferta ou materiais ainda não foram ouvidos, comece com `$brandfy-entrevista`. Quando a marca possui manual e ativos anteriores, use `$brandfy-diagnostico` para inventariá-los antes de redesenhar.

Use `$brandfy-builder` quando o trabalho abranger estratégia, voz, visual, ativos e manual. Um pedido de escopo menor deve chamar a especialidade correspondente. Por exemplo:

```
Use $brandfy-design-tokens para recompilar CSS, JSON e Tailwind a partir da
paleta aprovada em .brandfy/palette.json.
```

Confira a primeira estrutura

O setup prepara arquivos editáveis, não uma marca aprovada. Antes da entrevista, os documentos podem conter campos vazios e instruções. O resultado esperado é uma base organizada que preserve o trabalho existente e deixe as lacunas visíveis.

Não publique o conteúdo inicial de `brand/` como manual final. A publicação começa somente depois que as especialidades registrarem as fontes, os arquivos visuais forem abertos e a auditoria não apresentar reprovações.

Entrevista e diagnóstico

Inicie a entrevista

A entrevista deve ser conduzida com os responsáveis pela marca e com permissão para registrar as respostas. Inicialize a estrutura antes da conversa:

```
node .agents/skills/brandfy-entrevista/scripts/compile-interview.mjs --init
```

O comando cria `.brandfy/interview.json`. A skill conduz de três a cinco perguntas por etapa, começando pelo trabalho atual da empresa. Depois de cada bloco, ela resume fatos, hipóteses, escolhas e dúvidas para que o participante possa corrigir a síntese.

A entrevista cobre oferta, públicos, alternativas, posicionamento, personalidade, voz, direção visual, patrimônio existente, propriedade intelectual, aplicações e governança. Respostas desconhecidas continuam marcadas como desconhecidas, com uma pergunta e um responsável, em vez de receberem uma formulação inventada.

Valide a cobertura

Depois da conversa, execute:

```
node .agents/skills/brandfy-entrevista/scripts/compile-interview.mjs --check
```

O modo `--check` rejeita uma entrevista marcada como pronta quando faltam confirmações, campos obrigatórios ou etapas. Corrija o JSON com as respostas recebidas, preencha o nome do responsável e a data da confirmação, então rode o compilador sem argumentos:

```
node .agents/skills/brandfy-entrevista/scripts/compile-interview.mjs
```

O compilador atualiza blocos delimitados e preserva texto escrito fora deles. Ele produz a síntese da entrevista e leva conteúdo confirmado para o briefing, a estratégia, a voz e o registro jurídico.

Diagnostique uma marca existente

`$brandfy-diagnostico` abre os arquivos existentes em vez de confiar somente nos nomes. O inventário registra formato, dimensão, versão, cor, fonte, autoria, licença e uso observado. Ele compara o manual, o site, os templates e as respostas da entrevista para encontrar convergências e divergências.

O diagnóstico não move nem converte ativos. Seu resultado fica em `.brandfy/diagnostico.md` e indica quais skills precisam atuar. Um logo antigo com reconhecimento comprovado pode ser preservado, enquanto uma paleta sem contraste pode seguir para revisão de identidade visual e design tokens.

Leia o resultado com os responsáveis

A síntese compilada é uma base para as especialidades, não aprovação automática da estratégia. Releia os conflitos e as perguntas abertas antes de iniciar o desenho. Uma divergência sobre o público prioritário ou a promessa altera voz, logo, cor e aplicações, por isso deve permanecer visível até receber confirmação.

Construção do sistema de marca

Use o fluxo completo

Peça a coordenação das etapas quando a marca ainda precisa de base verbal, direção visual e arquivos:

```
Use $brandfy-builder para construir a marca completa a partir da entrevista confirmada e dos ativos inventariados.
```

O builder começa pelo setup, entrevista e diagnóstico conforme o estado encontrado. Depois, encaminha a estratégia, o naming quando necessário, o slogan, a voz, a identidade visual, a tipografia, o logo, os ativos, os tokens, as aplicações, o manual, o PDF e a auditoria.

Uma marca existente pode saltar uma etapa quando o diagnóstico aponta a fonte aprovada e o uso atual. O arquivo preservado continua sujeito à conferência de licença, acessibilidade e coerência com o restante do sistema.

Estratégia, naming e slogan

`$brandfy-estrategia` conecta público, necessidade, alternativas, promessa, diferença, provas e comportamento. Missão e promessa precisam refletir a capacidade atual. A visão pode apontar uma ambição futura, desde que não seja apresentada como estado já alcançado.

`$brandfy-naming` explora territórios diferentes e registra buscas indicativas de grafia, fonética, domínio, perfis sociais e marcas relacionadas. Essa pesquisa não substitui um parecer de propriedade intelectual nem permite prever o exame do INPI. O trabalho com uma opção que apresente conflito deve parar até a análise adequada.

`$brandfy-slogan` testa entendimento, verdade, pronúncia, uso junto ao logo e vida útil. A ausência de uma frase adequada é um resultado aceitável; um slogan fraco não deve ser mantido apenas para preencher o lockup.

Voz e tom

`$brandfy-voz` documenta como a marca explica, orienta, vende, cobra e reage a uma falha. Cada princípio registra intenção, mecanismo, limite, exemplo, contraexemplo e evidência. O tom muda conforme a situação, mas a relação com o leitor e a promessa permanecem reconhecíveis.

O arquivo `brand/voice.md` precisa ser específico o bastante para orientar uma página, um email, uma mensagem de suporte e um texto de interface. Quando dois redatores aplicam a regra de formas incompatíveis, faltam mecanismos ou exemplos.

Identidade visual e logo

`$brandfy-identidade-visual` traduz atributos em princípios de composição, cor, tipografia, fotografia, iconografia e motion. As rotas precisam diferir em estrutura, trazer vantagens e limitações e aparecer em mais de uma aplicação.

`$brandfy-logo` desenvolve o sistema aprovado, com símbolo, wordmark e lockups necessários. O SVG mestre deve usar formas vetoriais, `viewBox` e nomes previsíveis, sem depender de uma imagem raster ou de uma fonte sem licença. A seleção considera redução, uma cor, impressão, avatar, favicon, fundos claros e escuros.

Gates de aprovação

O fluxo interrompe a passagem para o visual quando nome, público, promessa ou restrições de uso continuam contraditórios. Em cada marco, o responsável deve conseguir explicar a alternativa escolhida, a evidência usada, a limitação aceita e o arquivo que registra a aprovação.

Logos, webfontes, tokens e templates

Exporte o sistema de logo

Mantenha os SVGs mestres em `brand/logo/svg/` e execute:

```
node .agents/skills/brandfy-ativos-logo/scripts/export-logo.mjs \  
  --input brand/logo/svg \  
  --output brand/logo/png \  
  --font brand/fonts/manrope-variable.ttf
```

O exportador produz PNGs a partir das fontes vetoriais e atualiza o manifesto quando configurado. Os sufixos `light` e `dark` indicam o fundo no qual cada arquivo será aplicado. Abra uma amostra de cada variante para conferir transparência, margem, proporção e leitura no tamanho mínimo.

Favicons e avatares pedem uma versão compacta. Uma assinatura horizontal reduzida até caber em 16 px perde leitura mesmo quando o arquivo continua tecnicamente válido.

O arquivo informado em `--font` mantém o wordmark consistente quando o SVG usa texto editável. O projeto pode omitir esse argumento quando todos os elementos do logo já estiverem convertidos em paths.

Prepare as webfontes

Preencha o manifesto de fontes com família, origem, licença, arquivos, pesos e fallbacks. Depois execute:

```
node .agents/skills/brandfy-tipografia-web/scripts/download-fonts.mjs \  
  --manifest .brandfy/fonts.json \  
  --output brand/fonts
```

O gerador baixa somente arquivos declarados, registra a licença e produz o CSS tipográfico. Use `WOFF2` e `font-display: swap` quando a família permitir. A ausência da webfont deve acionar o fallback sem esconder conteúdo nem desmontar a hierarquia.

Antes de versionar uma fonte, confirme que a licença permite redistribuição e hospedagem própria. Arquivos proprietários permanecem no serviço autorizado ou no ambiente licenciado.

Gere os design tokens

A paleta editável fica em `.brandfy/palette.json`. Ela separa famílias de cor das funções semânticas usadas por websites e sistemas:

```
node .agents/skills/brandfy-design-tokens/scripts/generate-tokens.mjs \
  --input .brandfy/palette.json \
  --output brand
```

O comando gera `global.css`, `tokens.json`, `tailwind-theme.js` e `accessibility.md`. Os tokens semânticos definem fundo, superfície, texto, texto secundário, borda, acento e foco nos modos light e dark. Cores de sucesso, aviso e erro podem ser adicionadas quando a interface realmente usa esses estados.

Importe `brand/global.css` no website. Em Tailwind CSS, aplique o objeto de `brand/tailwind-theme.js` em `theme.extend`. Prefira os nomes semânticos a valores hexadecimais copiados para componentes.

Instale templates de canais

O Brandfy inclui fontes SVG para Instagram, LinkedIn, email e YouTube:

```
node .agents/skills/brandfy-templates-canais/scripts/install-templates.mjs \
  --output brand/templates
```

Arquivos existentes são preservados. Use `--force` somente quando a fonte aprovada deve ser substituída conscientemente. Antes de publicar, aplique a paleta, as fontes e o logo da marca, remova as guias da exportação e confira a leitura no tamanho final.

O carrossel usa 1080 por 1350 px, a arte do LinkedIn usa 1200 por 1200 px, o cabeçalho de email é criado em 1200 por 400 px e a thumbnail do YouTube usa 1280 por 720 px. O banner do canal tem 2560 por 1440 px, mas textos e logos ficam na área central segura de 1546 por 423 px.

Manual editável e PDF da marca

O Markdown é a fonte

O manual canônico vive em `brand/README.md`. Ele liga a estratégia aos arquivos e explica como designers, redatores, desenvolvedores e agentes devem usar a marca. Corrija o Markdown e recompile o PDF; não faça uma correção isolada no arquivo binário.

Um manual completo cobre origem, estratégia, posicionamento, personalidade, voz, logo, cor, tipografia, imagens, aplicações, acessibilidade, licenças e governança. Os links relativos precisam apontar para arquivos que existam dentro da pasta da marca.

Compile o guia

Com Pandoc e WeasyPrint disponíveis no `PATH`, execute:

```
node .agents/skills/brandfy-guia-pdf/scripts/build-brand-guide.mjs \
  --project . \
  --input brand/README.md \
  --output brand/brand-guide.pdf \
  --brand-name "Nome da marca"
```

Antes da compilação, a skill copia CSS, template, Inter, Manrope e licenças para `brand/pdf/`. Arquivos locais existentes são preservados. O Pandoc converte o Markdown para HTML, cria o sumário e incorpora recursos locais. O WeasyPrint aplica o CSS e produz o PDF.

Para instalar o kit sem compilar:

```
node .agents/skills/brandfy-guia-pdf/scripts/build-brand-guide.mjs \
  --project . \
  --install-assets
```

Use `--force-assets` somente para atualizar deliberadamente a cópia local.

Abra o resultado

Uma compilação sem erro não comprova a qualidade visual. Abra a capa, o sumário, páginas com tabela, páginas com imagens e a última página. Confirme as fontes, a numeração, as quebras, a proporção dos logos e os links.

Quando uma imagem não aparece, confira o caminho relativo a `brand/README.md`. Quando a fonte não carrega, confirme a URL no CSS e a licença ao lado do arquivo. Tabelas extensas podem exigir uma redação mais compacta ou outra disposição, mas o Markdown continua sendo a origem.

Diferencie os dois PDFs

O PDF em `brand/brand-guide.pdf` é o manual da identidade de um projeto consumidor e é compilado de `brand/README.md` dentro desse projeto. O PDF em `brandfy/ebooks/` pertence ao repositório do Brandfy e é a edição portátil deste guia de uso. Eles possuem fontes, destinos e públicos diferentes, mas usam o mesmo `pdf-design-system: 1.0.0`.

Auditoria da marca

Execute a conferência automatizada

Depois de compilar o manual e gerar os ativos, rode:

```
node .agents/skills/brandfy-auditoria/scripts/audit-brand.mjs \
  --project . \
  --config .brandfy/config.yaml
```

O relatório padrão fica em `.brandfy/audit.md`. Ele separa reprovações, avisos, evidências encontradas e itens que dependem de inspeção humana. A skill não corrige silenciosamente um arquivo aprovado.

Confronte definição, fonte e aplicação

A auditoria compara o que o manual afirma com o arquivo fonte, a exportação e a aplicação observada. Uma paleta pode estar documentada corretamente e ainda falhar quando o template usa um valor antigo. Um PNG pode ter a dimensão prevista e ainda apresentar margem desigual ou transparência incorreta.

Abra pelo menos:

- as variantes light e dark do logo;
- o ícone no tamanho mínimo;
- uma peça de cada canal prioritário;
- uma página de interface nos dois temas;
- o manual em PDF;
- um texto produzido com o guia de voz.

Também confira a licença das fontes, a procedência de fotografias, o consentimento de imagem e o registro das pesquisas de naming. A busca de nome e domínio continua sendo indicativa.

Interprete o relatório

Uma reprovação impede a publicação do conjunto afetado. Um aviso registra uma correção necessária ou uma melhoria cuja consequência precisa ser avaliada. A inspeção humana deve acrescentar evidência ao relatório em vez de apenas marcar um item como concluído.

Depois de corrigir um arquivo, atualize o manifesto quando necessário, recompile o PDF e execute a auditoria outra vez. O resultado está pronto quando o manual, os ativos e as aplicações concordam e as pendências aceitas possuem responsável e motivo.

Uso da marca por agentes

Dê ao agente uma fonte única

O setup inclui no `AGENTS.md` um bloco que orienta o uso de `.brandfy/` e `brand/`. Preserve os marcadores porque uma execução posterior atualiza somente esse trecho. Instruções específicas do projeto continuam fora do bloco.

Antes de criar uma peça, o agente deve ler `.brandfy/config.yaml`, o manual e os arquivos ligados ao canal. Para uma arte social, isso inclui os tokens, o logo apropriado e o guia do template. Para um texto, inclui a estratégia, a voz e os exemplos do canal.

Escreva instruções verificáveis

Uma solicitação útil nomeia a skill, a entrada, o canal e a saída:

```
Use $brandfy-templates-canais para preparar um carrossel de Instagram a partir do template aprovado. Leia brand/README.md e brand/templates/README.md, preserve a zona segura, salve o SVG editável e abra o PNG final em 1080 × 1350.
```

O agente precisa relatar quais arquivos leu, quais arquivos criou, que conferências executou e o que permaneceu pendente. Uma resposta que apenas afirma fidelidade à marca sem apontar o manual e o arquivo resultante não oferece evidência suficiente.

Preserve ativos aprovados

Uma nova execução não deve redesenhar o logo, trocar fontes ou substituir um template aprovado para atender uma peça isolada. Quando a aplicação revela uma limitação do sistema, registre o achado e encaminhe a revisão à skill especialista.

Fotografias institucionais dependem de arquivo autorizado. Uma imagem gerada não substitui o retrato oficial de uma pessoa. Toda fonte, ilustração ou elemento externo precisa manter autoria, licença e procedência.

Confira a exportação

O arquivo editável e a exportação cumprem funções diferentes. O SVG permite revisão futura; o PNG mostra como a peça chega ao canal. Peça ao agente para abrir a exportação no tamanho final, testar contraste e confirmar que guias, placeholders e conteúdo reservado foram removidos.

Solução de problemas

A skill não aparece

Liste a origem e procure o arquivo instalado:

```
npx skills add promovaweb/brandfy --list  
find . -path '*/brandfy-setup/SKILL.md' -print
```

Quando o diretório não existe, repita a instalação na raiz do projeto. Quando ele existe em outro prefixo, ajuste os comandos diretos e reinicie o agente conforme a ferramenta em uso.

O setup alteraria arquivos

Execute o modo de conferência:

```
node .agents/skills/brandfy-setup/scripts/setup.mjs --project . --check
```

Abra o `AGENTS.md` e procure os dois marcadores do Brandfy. Um bloco incompleto ou duplicado precisa ser corrigido antes de outra execução. Preserve todo conteúdo autoral fora dos marcadores.

A entrevista não passa no check

Abra `.brandfy/interview.json` e leia as mensagens do compilador. Uma entrevista pronta exige confirmação, data, participantes, etapas concluídas e campos mínimos. Uma resposta ainda desconhecida deve aparecer na coleção de pendências com pergunta e responsável.

Não altere `status` para `ready` apenas para satisfazer o validador. Complete a conversa ou registre honestamente o que falta.

O PNG não foi gerado

Confirme o ImageMagick e a presença dos SVGs mestres:

```
magick -version  
find brand/logo/svg -type f -name '*.svg' -print
```

Abra o SVG que falhou e confira `viewBox`, formas vetoriais e referências externas. Uma fonte ausente ou uma imagem vinculada pode funcionar no editor e falhar no ambiente do gerador.

A webfont não carrega

Confira a URL registrada em `brand/fonts/fonts.css`, o nome do arquivo e o MIME type servido pelo website. Abra a página com a rede desabilitada para testar o fallback. Quando o arquivo não pode ser redistribuído, use o serviço licenciado em vez de copiá-lo para o repositório.

O contraste foi reprovado

Leia `brand/accessibility.md` e identifique a função semântica, o tema e a combinação exata. Uma cor aceita no logo pode ser inadequada para texto, controle ou indicador de foco. Corrija a paleta editável e gere novamente os tokens, em vez de aplicar um valor isolado no componente.

O PDF ficou sem imagens ou fontes

Execute o build na raiz do projeto e confira caminhos relativos ao Markdown. Verifique também Pandoc e WeasyPrint:

```
pandoc --version  
weasyprint --version
```

Depois da correção, apague somente arquivos temporários declarados pelo compilador e gere o PDF outra vez. Nunca edite o PDF como fonte.

A auditoria ainda apresenta avisos

Abra `.brandfy/audit.md` e relacione cada aviso ao arquivo observado. Alguns itens exigem inspeção humana, como legibilidade no feed, procedência de uma foto ou coerência do tom. Registre a evidência da conferência e execute a auditoria novamente quando houver alteração técnica.

Dados para um relato de falha

Inclua a versão do Node.js, o sistema operacional, o caminho da skill, o comando executado, a mensagem completa e a lista dos arquivos esperados. Remova tokens, informações pessoais, entrevistas reservadas e ativos que não podem ser compartilhados.

Guia de desenvolvimento

Este percurso explica como o Brandfy transforma uma especialidade de branding em uma skill instalável. Ele se destina a mantenedores do repositório, desenvolvedores de geradores, revisores de contratos e responsáveis por uma publicação.

Leia a arquitetura antes de modificar uma skill. Depois, consulte o contrato das skills, os geradores, os testes e o processo de contribuição.

Autoridades técnicas

FONTE	AUTORIDADE
<code>skills/*/SKILL.md</code>	Procedimento público de cada especialidade
<code>skills/*/scripts/</code>	Interface executável dos geradores
<code>skills/*/references/</code>	Parâmetros técnicos carregados sob demanda
<code>skills/*/assets/</code>	Arquivos-base distribuídos com a skill
<code>tests/</code>	Comportamentos automatizados
<code>scripts/validate.mjs</code>	Estrutura pública e metadados
<code>AGENTS.md</code>	Regras do repositório e bloco canônico do consumidor

Uma página desta pasta explica o código, mas não cria uma interface nova. Toda mudança de comando precisa começar na implementação e terminar com teste e documentação equivalentes.

Validação rápida

Na raiz do Brandfy:

```
npm test
```

O comando executa os testes de setup, entrevista e geradores, depois confere a estrutura das skills. Alterações documentais também precisam passar pelo lint de Markdown e pelo build do ebook quando atingirem `docs/`.

Arquitetura do Brandfy

Biblioteca instalada no projeto consumidor

O Brandfy distribui conhecimento e automação como diretórios independentes dentro de `skills/`. O gerenciador `skills add` copia as especialidades selecionadas para o projeto consumidor. A execução acontece no repositório da marca, por isso os scripts resolvem entradas e saídas a partir do diretório informado, não a partir de dados privados guardados no Brandfy.

ORIGEM	PASSAGEM	DESTINO
Repositório Brandfy	<code>skills add</code> seleciona e copia as especialidades	Skills instaladas no projeto consumidor
Skills instaladas	O agente lê método, referências e entradas locais	<code>.brandfy/</code> recebe pesquisa e acompanhamento
Skills instaladas	Os geradores escrevem fontes e derivados aprovados	<code>brand/</code> recebe manual e ativos
<code>brand/</code>	A auditoria confronta definição, fonte e aplicação	<code>.brandfy/audit.md</code> registra o resultado

`.brandfy/` recebe configuração, entrevista, briefing e relatórios. `brand/` recebe o manual, os ativos e as saídas compiladas. O agente lê as duas fontes, mas não mistura trabalho reservado com arquivos destinados a publicação.

CLI e gerenciador de skills

O pacote npm vive em `cli/` e expõe o comando `brandfy`. Essa camada resolve a versão empacotada do `skills`, executa `skills add` no projeto consumidor e localiza os scripts instalados em `.agents/skills/`, `.codex/skills/` ou `.claudeskills/`.

O CLI não incorpora uma segunda cópia dos geradores. `brandfy pdf` e `brandfy audit` executam os arquivos das skills instaladas, mantendo uma única fonte para o comportamento. O framework, o pacote npm e o ebook usam a mesma SemVer, conferida por `npm run release:check`.

Tipos de skill

As 18 skills cumprem quatro papéis:

PAPEL	SKILLS
Coordenação	<code>brandfy-setup</code> , <code>brandfy-builder</code>

PAPEL	SKILLS
Descoberta e base verbal	<code>brandfy-entrevista</code> , <code>brandfy-diagnostico</code> , <code>brandfy-estrategia</code> , <code>brandfy-naming</code> , <code>brandfy-slogan</code> , <code>brandfy-voz</code>
Sistema visual	<code>brandfy-identidade-visual</code> , <code>brandfy-logo</code> , <code>brandfy-tipografia-web</code> , <code>brandfy-design-tokens</code>
Produção e conferência	<code>brandfy-ativos-logo</code> , <code>brandfy-aplicacoes</code> , <code>brandfy-templates-canais</code> , <code>brandfy-manual</code> , <code>brandfy-guia-pdf</code> , <code>brandfy-auditoria</code>

As skills de coordenação fazem handoff para uma especialidade e retomam o plano quando o arquivo esperado existe. Elas não duplicam o método detalhado pela skill chamada.

Limites de escrita

Os geradores são idempotentes por contrato. O setup atualiza somente o bloco delimitado no `AGENTS.md` ; o compilador de entrevista atualiza blocos `brandfy:interview` ; o instalador de templates preserva arquivos existentes sem `--force` . Exportações derivadas podem ser atualizadas quando sua fonte correspondente mudou.

Um script não deve mover nem reescrever um ativo aprovado sem uma opção explícita e documentada. O diagnóstico e a auditoria são essencialmente leitores: eles registram achados em relatório e deixam a correção para a especialidade apropriada.

Dependências externas

O núcleo dos geradores usa APIs nativas do Node.js. O CLI depende do pacote `skills` , com versão fixa no lock. ImageMagick entra na rasterização dos logos e templates. Pandoc e WeasyPrint entram na compilação de PDFs. O projeto mantém os três últimos programas fora das dependências npm porque eles possuem ciclos de instalação próprios.

Contrato das skills

Estrutura mínima

Cada diretório em `skills/` contém `SKILL.md` e `agents/openai.yaml`. Scripts, referências e arquivos-base entram somente quando a especialidade precisa deles:

```
skills/brandfy-exemplo/  
├─ SKILL.md  
├─ agents/  
│   └─ openai.yaml  
├─ assets/  
├─ references/  
└─ scripts/
```

`SKILL.md` começa com frontmatter que declara `name` e `description`. O nome precisa corresponder ao diretório. A descrição deve informar a tarefa e a sinalização de uso com extensão suficiente para o agente selecionar a skill.

Protocolo operacional

Toda skill documenta cinco partes: plano e progresso, fontes de verdade, escopo e idempotência, validação e resumo final. Depois, ela explica o fluxo próprio e o raciocínio do especialista.

A instrução precisa levar o agente a comparar alternativas e a fundamentar uma recomendação com evidência ou parâmetro verificável. Adjetivos vagos não substituem a explicação. Uma direção descrita como “moderna” deve mostrar a composição, o uso, a limitação e a relação com a estratégia.

Metadados para o agente

`agents/openai.yaml` mantém a interface mostrada pelo agente e um `default_prompt` que cita a skill no formato `$brandfy-nome`. O validador confere essa referência para impedir um diretório instalável cuja interface não chama a própria especialidade.

Referências e arquivos-base

Uma referência contém parâmetros que a skill precisa ler para executar a tarefa. Ela não deve repetir regras gerais que já estão no `SKILL.md`. Um arquivo em `assets/` é distribuído como fonte, exemplo ou template e nunca deve ser confundido com resultado aprovado de uma marca.

Scripts ficam próximos da skill porque são parte da mesma interface pública. Os argumentos precisam aceitar caminhos explícitos, produzir mensagens em Português do Brasil e sair com código diferente de zero quando a saída não puder ser comprovada.

Compatibilidade com o Skill Creator

Além de `npm test`, cada skill alterada deve passar pelo `quick_validate.py` fornecido pelo Skill Creator. O teste local do Brandfy confere regras próprias, enquanto o validador externo confere o formato aceito pelo ecossistema.

Contrato dos artefatos

Configuração

`.brandfy/config.yaml` informa o nome do projeto, o diretório da marca, o modo de trabalho, o idioma, o público principal e o estado. Scripts que aceitam `--config` devem resolver caminhos a partir da raiz indicada por `--project`.

O formato YAML atual é deliberadamente simples. Uma expansão precisa preservar campos desconhecidos e incluir teste para projetos já configurados.

Arquivos de trabalho

O setup pode preparar:

```
.brandfy/
├─ config.yaml
├─ brief.md
├─ interview.json
├─ interview-summary.md
├─ diagnostico.md
├─ audit.md
├─ review.md
├─ interviews/
└─ evidence/
```

Nem todos os arquivos aparecem na primeira execução. A skill responsável cria o documento quando existe informação para registrar. Dados pessoais desnecessários e ativos sem permissão não devem entrar em evidências.

Arquivos finais

O contrato completo de `brand/` admite:


```
brand/
├─ README.md
├─ strategy.md
├─ voice.md
├─ legal.md
├─ global.css
├─ tokens.json
├─ tailwind-theme.js
├─ accessibility.md
├─ manifest.json
├─ logo/
├─ favicon/
├─ fonts/
├─ colors/
├─ templates/
├─ social/
├─ print/
├─ photography/
└─ archive/
```

Uma marca não precisa preencher diretórios sem uso previsto. O manifesto lista as exportações existentes com caminho, formato, dimensão, função e hash. Uma entrada que aponta para arquivo ausente reprova a auditoria.

Fontes e derivados

SVGs mestres, paleta e Markdown são fontes editáveis. PNGs, tokens gerados e PDFs são derivados reproduzíveis. A revisão deve começar na fonte e executar o gerador correspondente, porque uma alteração direta no derivado desaparece no próximo build.

Substituições importantes podem preservar a versão anterior em `brand/archive/`. O arquivo arquivado precisa manter origem e data suficientes para investigação, sem continuar exposto como opção oficial.

Geradores e interfaces de linha de comando

Setup

```
node skills/brandfy-setup/scripts/setup.mjs --project <diretório> [--check]
```

Sem `--check`, prepara `.brandfy/`, `brand/` e o bloco do consumidor no `AGENTS.md`. Com `--check`, compara o estado esperado sem escrever.

Entrevista

```
node skills/brandfy-entrevista/scripts/compile-interview.mjs \  
  [--project <diretório>] [--input <arquivo>] [--init | --check]
```

`--init` cria o JSON inicial. `--check` valida cobertura e confirmações. Sem esses modos, o script compila as respostas e atualiza os blocos delimitados nos documentos derivados.

Logos

```
node skills/brandfy-ativos-logo/scripts/export-logo.mjs \  
  --input <diretório-svg> \  
  --output <diretório-png> \  
  [--manifest <arquivo>] \  
  [--font <arquivo-ttf-ou-otf>]
```

O exportador depende do ImageMagick. A interface precisa preservar as fontes SVG e relatar qualquer variante que não possa ser rasterizada.

Quando um SVG contém texto editável, `--font` informa a fonte usada na rasterização. O padrão é `brand/fonts/manrope-variable.ttf`; arquivos formados somente por paths não dependem desse argumento.

Design tokens

```
node skills/brandfy-design-tokens/scripts/generate-tokens.mjs \  
  --input <palette.json> \  
  --output <diretório>
```

O gerador lê famílias e funções light/dark, escreve CSS, JSON, tema Tailwind e relatório de contraste. A saída CSS inclui troca automática por `prefers-color-scheme` e seletores explícitos por `data-theme`.

Webfontes

```
node skills/brandfy-tipografia-web/scripts/download-fonts.mjs \  
  --manifest <fonts.json> \  
  --output <diretório>
```

O manifesto determina URLs, hashes quando disponíveis, licenças e fallbacks. O script não deve baixar uma família cuja redistribuição não esteja autorizada.

Templates

```
node skills/brandfy-templates-canais/scripts/install-templates.mjs \  
  --output <diretório> [--force]
```

Sem `--force`, arquivos existentes permanecem intactos. A opção explícita substitui os templates conhecidos pelo conteúdo distribuído com a skill.

Manual em PDF

```
node skills/brandfy-guia-pdf/scripts/build-brand-guide.mjs \  
  --project . \  
  --input <manual.md> \  
  --output <manual.pdf>
```

O script instala o kit versionado em `brand/pdf/`, cria HTML temporário com Pandoc e PDF com WeasyPrint. Caminhos de recursos são resolvidos a partir do projeto e do diretório do Markdown. `--install-assets` copia somente o kit; `--force-assets` substitui conscientemente os arquivos locais.

Auditoria

```
node skills/brandfy-auditoria/scripts/audit-brand.mjs \  
  --project <diretório> \  
  [--config <arquivo>] \  
  [--report <arquivo>]
```

A auditoria lê a configuração, verifica arquivos e grava o relatório. Ela deve continuar sem escrita nos ativos, inclusive quando encontra reprovações.

Testes e validação

Suíte principal

Execute na raiz do Brandfy:

```
npm test
```

O script roda os testes dos geradores, a suíte em `cli/tests/` e os validadores estruturais. Os testes do CLI injetam um executor falso, portanto não usam a rede nem modificam skills instaladas na máquina.

Cobertura atual

`tests/setup.test.mjs` cria um projeto temporário, executa o setup duas vezes e confirma que o conteúdo autoral do `AGENTS.md` foi preservado.

`tests/interview.test.mjs` inicializa uma entrevista, comprova falhas de cobertura, compila uma versão confirmada e verifica a idempotência dos blocos gerados.

`tests/generators.test.mjs` gera tokens em diretório temporário e instala os templates sem substituir arquivos. Novos geradores devem receber um teste que use entradas pequenas e confira conteúdo, não somente a presença da saída.

`cli/tests/cli.test.mjs` confere ajuda, versão, argumentos repassados ao `skills`, setup, diagnóstico e compilação de PDF. O tarball ainda precisa ser instalado em um diretório temporário durante a preparação da release, porque essa verificação exerce o binário exatamente como o npm o distribuirá.

Validador estrutural

`scripts/validate.mjs` percorre todas as skills e confere:

- existência de `SKILL.md` e `agents/openai.yaml`;
- correspondência entre frontmatter e diretório;
- extensão da descrição;
- ausência de marcadores `TODO`;
- presença do protocolo de plano e validação;
- postura de especialista;
- chamada `$brandfy-*` nos metadados;
- igualdade do bloco do consumidor com o trecho canônico de `AGENTS.md`;
- presença dos scripts essenciais.

Documentação e ebook

O comando abaixo confere a ordem das páginas, o manifesto, os hashes, a estrutura XML do EPUB e a leitura básica do PDF:

```
npm run ebook:verify
```

Quando `docs/` muda, incremente `ebooks/VERSION`, execute `npm run ebook` e rode a verificação. A versão precisa ser igual à de `package.json` e `cli/package.json`.

Release

O comando abaixo confere a versão única, a entrada correspondente no changelog e os arquivos do pacote público:

```
npm run release:check
```

O processo completo de publicação está em `RELEASING.md`.

Inspeção humana

Testes não detectam um logo visualmente deformado, uma tabela quebrada no PDF ou um capítulo difícil de seguir. Abra amostras dos ativos e leia a documentação completa depois dos validadores formais.

Contribuição e manutenção

Prepare a alteração

Leia `AGENTS.md`, confira o estado do Git e identifique os arquivos que constituem a interface pública. Uma mudança em argumento de script normalmente atinge o próprio script, o `SKILL.md`, esta documentação e pelo menos um teste.

Mantenha comentários, mensagens e documentação em Português do Brasil. Termos técnicos permanecem em inglês quando essa é a forma reconhecida pelo ecossistema.

Crie ou altere uma skill

Uma nova skill precisa representar uma especialidade que não cabe de forma coesa em uma skill existente. Prepare `SKILL.md`, metadados e somente os recursos necessários. Não copie todo o manual para `references/`; carregue a fonte oficial do projeto consumidor.

Scripts devem aceitar caminhos explícitos, preservar trabalho autoral e fornecer uma forma de conferência quando houver escrita relevante. Documente dependências externas e mensagens de falha.

Documente o comportamento

Atualize o guia do usuário quando a mudança altera instalação, sequência de uso, comando ou arquivo produzido. Atualize o guia técnico quando muda a arquitetura, o contrato ou a manutenção.

Todas as páginas entram uma única vez em `docs/reading-order.txt`. Uma página esquecida reprova o build do ebook para impedir a publicação de uma edição incompleta.

Valide antes da publicação

Execute:

```
npm test
npm run ebook
npm run ebook:verify
npm run release:check
npm run cli:pack
```

Rode também o `quick_validate.py` do Skill Creator para cada skill alterada. Depois, confira o diff, abra o PDF e o EPUB e leia os trechos afetados.

Versione a release

O framework, o CLI e o ebook usam a mesma SemVer. Aplique a versão com:

```
npm run version:set -- 1.2.0
```

Uma correção compatível aumenta `PATCH`, uma funcionalidade nova aumenta `MINOR`, e uma mudança incompatível aumenta `MAJOR`. O processo de commit, tag, publicação npm e GitHub Release está em `RELEASING.md`.